Segundo Parcial

Practica 04

CHAT CONCURRENTE

REPORTE

CONCURRENCIA Y PARALELISMO

INGENIERIA EN TECNOLOGIA DE SOFTWARE

ALAN CAMPOS SUAREZ

UNIVERSIDAD AUTONOMA DE CAMPECHE

DEL SERVIDOR ECO AL CHAT GRUPAL

En esta práctica modifiqué el servidor y el cliente de la práctica anterior para convertirlos en un chat grupal. Antes, el servidor solo respondía al cliente que le escribía; ahora cada mensaje que llega se reenvía a todos los demás usuarios conectados.

El servidor (ParallelServer) escucha en el puerto 8080 y por cada conexión nueva entrega el socket a un hilo de un pool creado con `newCachedThreadPool()`. Así el ciclo principal queda libre para aceptar al siguiente cliente sin esperar.

El cliente (ConcurrentClient) pide un nombre de usuario, y después puede enviar mensajes y recibir los de los demás al mismo tiempo.

INICIO DE SESIÓN Y NOMBRES ÚNICOS

Para saber quién escribe cada mensaje agregué un login. El servidor valida el nombre con una expresión regular (de 1 a 20 caracteres: letras, números, guion o guion bajo) y revisa que nadie más lo esté usando.

Para esto último guardo a los usuarios en un `ConcurrentHashMap` y uso `putIfAbsent`, que es atómico: si dos clientes piden el mismo nombre al mismo tiempo, solo uno lo consigue. Los nombres no distinguen mayúsculas.

Si el nombre no sirve, el servidor responde `ERROR:motivo` y el cliente vuelve a preguntar; si sirve, responde `OK`.

```
if (usuarios.putIfAbsent(nombre.toLowerCase(), out) != null) {
    out.println("ERROR:El nombre '" + nombre + "' ya está en uso.");
    continue;
}
out.println("OK");
```

BROADCAST Y CONCURRENCIA

El mismo mapa de usuarios sirve como lista de destinatarios: el método `broadcast` recorre el mapa y escribe el mensaje en el canal de cada usuario, menos el del remitente.

En el cliente, mientras el hilo principal espera lo que escribo con el teclado, un hilo receptor (daemon) lee lo que llega del servidor y lo imprime. Sin ese hilo, los mensajes de los demás solo aparecerían después de escribir algo.

```
private static void broadcast(String mensaje, String claveRemitente) {
  for (Map.Entry<String, PrintWriter> e : usuarios.entrySet())
    if (!e.getKey().equals(claveRemitente))
      e.getValue().println(mensaje);
}
```

```
Thread receptor = new Thread(() -> {
  while ((linea = input.readLine()) != null)
    System.out.print("\r" + linea + "\n> ");
});
receptor.setDaemon(true); receptor.start();
```

PROTOCOLO DE COMUNICACIÓN

Cliente y servidor se comunican con líneas de texto sobre TCP (puerto 8080). Estos son los mensajes que intercambian:

Envía	Mensaje	Significado
Cliente	`nombre`	Intento de login
Servidor	`OK`	Nombre aceptado
Servidor	`ERROR:motivo`	Nombre inválido o en uso
Servidor	`Bienvenido, ...`	Saludo con usuarios conectados
Cliente	`texto`	Mensaje para los demás
Servidor	`nombre: texto`	Mensaje de otro usuario
Servidor	`*** X se unió/salió ***`	Aviso de presencia
Cliente	`salir`	Terminar la sesión

CASOS DE PRUEBA

Login válido: al escribir un nombre correcto, el servidor responde OK y da la bienvenida.

Nombre inválido: con espacios, símbolos o más de 20 caracteres, el servidor responde ERROR y el cliente vuelve a pedirlo.

Nombre repetido: un segundo cliente con el mismo nombre (aunque cambien las mayúsculas) es rechazado.

Chat grupal: con tres clientes conectados, lo que escribe uno lo reciben los otros dos, pero no el remitente.

Entrada y salida: al conectarse o desconectarse un usuario, los demás reciben un aviso.

Comando salir: el cliente recibe la despedida, se cierra su conexión y el servidor libera su nombre.

Cierre abrupto o inactividad: si el cliente se cae, o pasa 180 segundos sin escribir, el servidor cierra la conexión y avisa al grupo.

MECANISMO DE SALIDA LIMPIA

Cuando escribo `salir`, el servidor se despide, cierra la conexión, quita al usuario del mapa y avisa al grupo. Si un cliente se cae sin avisar, o pasa 180 segundos sin escribir, el bloque `finally` hace la misma limpieza.

Así el nombre queda libre para otro usuario y ningún cliente se queda en la lista sin estar realmente conectado.

```
} finally {
    if (nombre != null) {
        usuarios.remove(nombre.toLowerCase());
        broadcast("*** " + nombre + " salió del chat ***", clave);
    }
}
```

CONCLUSION

En esta práctica aprendí a convertir un servidor concurrente en un chat de varios usuarios. Entendí por qué se necesita un hilo por cliente en el servidor y un hilo receptor en el cliente, y la importancia de usar estructuras seguras para hilos, como `ConcurrentHashMap`, junto con operaciones atómicas para evitar que dos usuarios tengan el mismo nombre.

También vi que hay que liberar siempre los recursos cuando un cliente se va, ya sea con `salir`, por inactividad o por una caída.